GOLD: Writing Classes (Mon Sep 21)

Designing a Class

Designing a class: fields + methods

Last week we used classes that other people wrote. Today you start to write your own.

Pick a real-world thing.

A class captures it (**abstraction**) in two categories:

- **Fields** are its **attributes** (what it knows):
  - a `Wheel`'s `radius`, its `material`
  - a `Student`'s `name` and `major`

- **Methods** are its **operations** (what it *does*):
  - `getCircumference()`, `setRadius(...)`
  - for a student: `declareMajor()` or `checkDegreeRequirements()`

```
class Wheel {
    double radius;      // field
    String material;    // field
    // ... methods go here ...
}
```

Our running example today: the **`Wheel`**.

A class is not a program

A class only *describes* a kind of thing. Something still has to **make** a Wheel and use it, and that is the job of your `main` method.

On these slides the program and the class share one snippet so that you can press Run.

The class that holds `main` always comes first:

```
class Main {
    void main() {
        Wheel bicycle = new Wheel();
        bicycle.radius = 13.75;
        bicycle.material = "rubber";
        IO.println(bicycle.material + " wheel, radius " + bicycle.radius);
    }
}

class Wheel {
    double radius;      // field
    String material;    // field
    // ... methods go here ...
}
```

`class Main { … }` is what Java has been quietly wrapping around all of our programs.

Now that a file holds two classes, we write it ourselves.



In a real project (and in the labs) each class lives in **its own file, named after the class**, and is marked `public`: class `Wheel` goes in `Wheel.java`.

Today's take-home problem follows this pattern with `Main.java` and `MenuItem.java`.

Fields

Fields: what an object knows

A **field** is a variable declared inside the class, outside any method.

Each `Wheel` object carries its own `radius` and `material`.

Two `Wheel` objects can hold different values in the same fields, because they are distinct objects, stored in different places in memory:

```
class Main {
    void main() {
        Wheel bicycle = new Wheel();
        Wheel wagon = new Wheel();
        bicycle.radius = 13.75;
        wagon.radius = 5.0;

        IO.println(bicycle.radius);     // 13.75
        IO.println(wagon.radius);       // 5.0    each Wheel has its own
        IO.println(wagon.material);     // null   never set: fields start at a default
    }
}

class Wheel {
    double radius;      // each Wheel has its own
    String material;    // each Wheel has its own
}
```

Instance vs static fields

```
public class Wheel {
    double radius;              // instance field: one PER object
    static int wheelCount;      // static field: ONE shared by all objects
}
```

- **Instance (non-static):** each object gets its own.
  - `bicycle.radius` != `wagon.radius` in general
- **Static:** a single value shared by the whole class.

    - Good for **counts**
    - and **constants**
    - `Math.PI` is a static field.

Fields start at a default

A new object's fields begin at the default for their type, exactly like the slots of a new array:

- `int`, `double`: `0`, `0.0`
- `boolean`: `false`
- any reference type, such as `String`: `null`

So, `wagon.material` is `null` until someone sets it.

- Calling a method on it, such as `wagon.material.length()`, is a `NullPointerException`.

- Setting the fields properly at birth is our next job.

Constructors

Constructors set up a new object

A **constructor** runs automatically when you say `new`. You called one on Friday: `new Random()`.

Its job: initialize the fields of the fresh object.

```
class Main {
    void main() {
        Wheel bicycle = new Wheel(27.5, "rubber");
        IO.println(bicycle.radius);        // 27.5
        IO.println(bicycle.material);      // rubber
    }
}

class Wheel {
    double radius;
    String material;

    // constructor: name == class, no return type
    Wheel(double radius, String material) {
        this.radius = radius;
        this.material = material;
    }
}
```

A constructor looks a lot like a method definition, **but:**

- it has the **same name as the class**,
- and **no return type** (not even `void`).

What `new` actually does

When we run `new Wheel(27.5, "rubber")`:

1. **Java allocates** a new `Wheel` object in memory (its fields exist, at their defaults)
2. it **runs the constructor** with `radius = 27.5`, `material = "rubber"`
3. it **returns a reference** to the finished object, which gets stored in `bicycle`

`this`: the object we're inside

Last time: we saw that `isEven(8)` was really `this.isEven(8)`.

Every non-static method is called **on an object** (`s.length()`, `wagon.getRadius()`),

and inside that method or constructor, **`this`** refers to that object:

```
Wheel(double radius, String material) {
    this.radius = radius;        // this.radius = the FIELD
    this.material = material;    // radius (no this) = the PARAMETER
}
```

Using `this` is often optional, but:

- It is **required** when a parameter has the same name as a field:
- `this.radius` means "the field of *this* object," not the parameter.

Two Constructor Traps

Two constructor traps

Trap 1: the constructor you didn't write

If a class has **no** constructor at all, Java supplies one for free:

- it takes no arguments and leaves every field at its default.
- That is how `new Wheel()` worked before.

The moment you write **any** constructor, the free one disappears:

```
class Wheel {
    double radius;
    String material;

    Wheel(double radius, String material) {
        this.radius = radius;
        this.material = material;
    }
}

// elsewhere:
Wheel w = new Wheel();      // used to work. Now:
```

```
error: constructor Wheel in class Wheel cannot be applied to given types;
  required: double,String
  found:    no arguments
```

Read: "the only way to build a Wheel needs a double and a String."

If you still want `new Wheel()`, write that constructor too.

Trap 2: the missing `this`

Pause and predict (poll)

```
class Main {
    void main() {
        Wheel w = new Wheel(27.5);
        IO.println(w.radius);
    }
}

class Wheel {
    double radius;

    Wheel(double radius) {
        radius = radius;
    }
}
```

What prints?

- **A.** `27.5`
- **B.** `0.0`
- **C.** `null`
- **D.** Compile error

Methods

Methods: name an operation

We have been writing methods already. Now we will put methods **inside a class**.

A **method** is a named, reusable block of code that may take inputs and return a value:

```
return_type methodName(parameter_list) {
    statements;
    return value;      // if return_type isn't void
}
```

Why methods?

- **Reusability** (write once, call many),
- **readability** (a name for an idea), and
- **modularity** (write and test in pieces).

Parameters vs arguments

```
int calc(double x, int y) {     // x and y are PARAMETERS
    return y + (int) Math.round(x);
}

void main() {
    int sum = calc(21.46, 15);      // 21.46 and 15 are ARGUMENTS
    IO.println(sum);
}
```

- **Parameter:** a variable in the method's declaration (`x`, `y`).
- **Argument:** the actual value you pass in at the call (`21.46`, `15`).
- Argument values are **copied** into parameter variables, matched by position and type.

- Argument values to a method must match the number and types of parameters of the method

Flow of control: what happens on a call

```
int sum = calc(21.46, 15);    // 1. copy args → params;
                              // 2. jump into calc
```

1. **Before:** argument values are copied into the parameters.
2. The caller is suspended; the method runs top to bottom until `return` (or the end).
3. **After:** the returned value replaces the call; the caller resumes.

Basic Methods: Accessors and mutators

With fields, the standard methods are **get** (read) and **set** (write):

```
double getRadius() {              // accessor — reads a field
    return radius;
}

void setRadius(double radius) {   // mutator — changes a field
    this.radius = radius;
}
```

- **Accessor** (`getX`) hands back a field's value.
- **Mutator** (`setX`) takes a new value and stores it.
- Why bother with getters and setters, instead of touching the field directly? More about that next time...

Example: accessors and mutators in a real class. Not every method is a `getX` or a `setX`:

```
class Main {
    void main() {
        Wheel wagon = new Wheel(5.0);
        IO.println(wagon.getRadius());             // 5.0
        wagon.setRadius(6.5);
        IO.println(wagon.getRadius());             // 6.5
        IO.println(wagon.getCircumference());      // about 40.84
    }
}

class Wheel {
    double radius;

    Wheel(double radius) {
        this.radius = radius;
    }

    double getRadius() {                  // accessor: reads a field
        return radius;
    }

    void setRadius(double radius) {       // mutator: changes a field
        this.radius = radius;
    }

    double getCircumference() {           // real behavior: computes from the field
        return 2 * Math.PI * radius;
    }
}
```

Style: The best practice a Javadoc block on every class and on every method, with an `@param` line for each parameter and an `@return` line when the method returns a value.

Our style checker will only enforce one Javadoc per file

Example:

```java
/**
 * Creates a menu item.
 *
 * @param name the name of the item
 * @param price the price in dollars
 */
public MenuItem(String name, double price) {
    this.name = name;
    this.price = price;
}

/**
 * Reports the current price of this item.
 *
 * @return the price in dollars
 */
public double getPrice() {
    return price;
}
```

The take-home starter file already has these blocks written, so you can see the pattern in a complete class.

Variable Vocabulary

Java's variable vocabulary

Important terms:

Term	What it is
Instance variable	a non-static field (one per object)
Class variable	a static field (one per class)
Local variable	a variable declared inside a method or block
Parameter	a variable in a method's declaration, filled in at call time

You will see all four on lab handouts and in error messages.

Example:

```
class Wheel {
    double radius;                  // instance variable: one per Wheel
    static int wheelCount;          // class variable:    one for the whole class

    void grow(double amount) {      // amount: a parameter
        double bigger = radius + amount;    // bigger: a local variable
        radius = bigger;
    }
}
```

- **Fields** live as long as their object (or, if static, as long as the program).
- **Parameters and locals** live only while the method runs.

Pause and predict (poll)

```
class Car {
    static int carCount;
    String color;
    int year;

    void setColor(String newPaint) {
        String paintColor = newPaint.toLowerCase();
        this.color = paintColor;
    }
}
```

Which of these is a **class variable**?

- **A.** `color`
- **B.** `carCount`
- **C.** `newPaint`
- **D.** `paintColor`

Static Fields: Counter

Problem: Counter

Let's make the shared-versus-own idea concrete.

We want a class where:

- each object has its **own** counter `x`, and
- there is **one** counter `y` shared by all objects.

Then create two objects, change the counters, and watch what happens. Predict the output before you run it:

```
class Main {
    void main() {
        Counter alice = new Counter();
        Counter bob = new Counter();

        alice.x = 10;
        alice.y = 42;
        bob.x = 50;
        bob.y = 99;         // writes the SAME y

        IO.println("alice.x=" + alice.x + " alice.y=" + alice.y);
        IO.println("bob.x=" + bob.x + " bob.y=" + bob.y);
    }
}

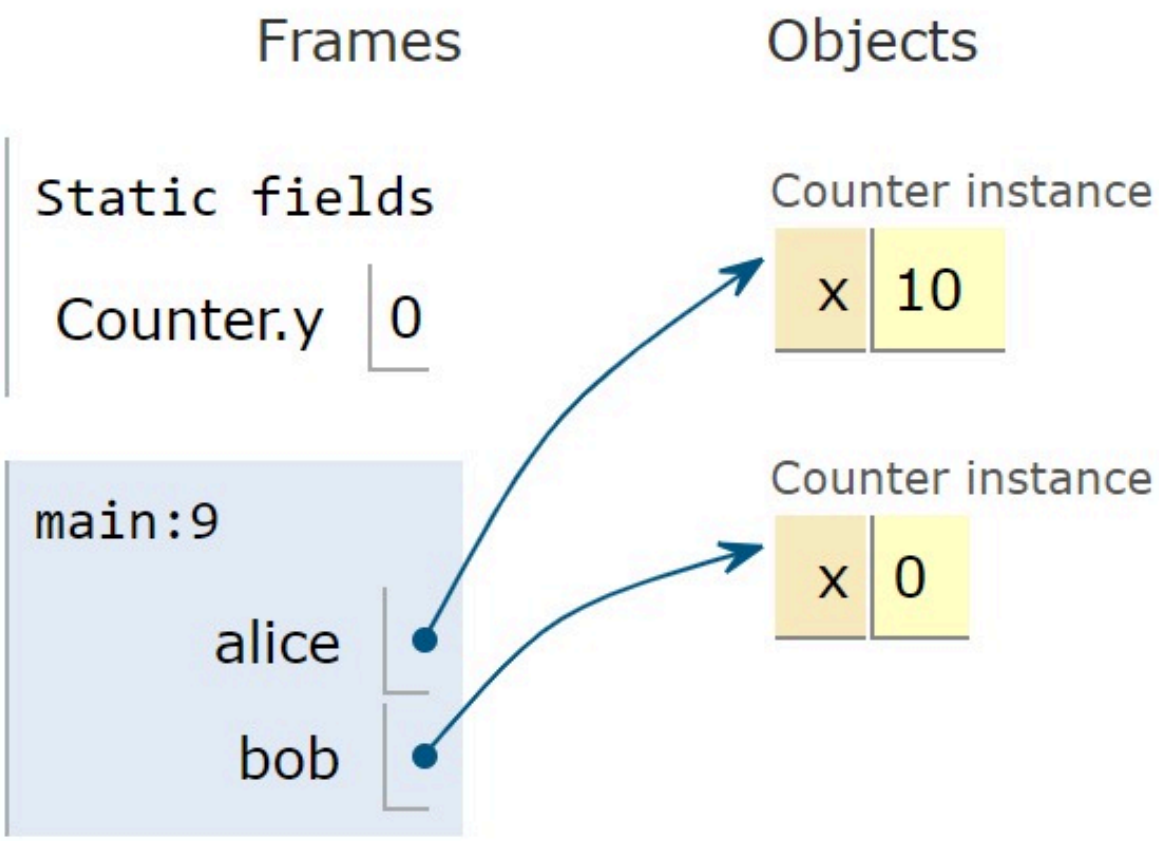
class Counter {
    int x;              // instance: one per object
    static int y;       // static: one shared by all
}
```

▶ Expand

Counter: the memory picture

```
alice.y++;        // y is now 100
Counter.y++;      // y is now 101
```

Both `alice` and `bob` see `y = 101`, because there is only one `y`.



Style note: `alice.y` and `bob.y` both compile, but the best practice is to reach a static through the **class name** (`Counter.y`), so that readers can see it is shared.

Pause and predict

```
class Main {
    void main() {
        Thing thing1 = new Thing();
        Thing thing2 = new Thing();
        for (int j = 0; j < 3; j++) {
            thing1.i++;
            thing1.s++;
            thing2.i++;
            thing2.s++;
        }
        IO.println(thing1.i + thing1.s + thing2.i + thing2.s);
    }
}

class Thing {
    static int s = 0;
    int i = 0;
}
```

What number is printed?

- **A.** 9
- **B.** 12
- **C.** 16
- **D.** 18
- **E.** 24

Static vs Non-static Methods

static vs non-static methods

We've used both kinds of methods before: `Math.pow(2, 5)` is called on the class, `s.length()` on an object.

Now we'll look at how to write both.

```
class Main {
    void main() {
        Wheel a = new Wheel(1.0);
        Wheel b = new Wheel(2.0);
        IO.println(b.getCircumference());     // instance: needs an object; uses b's r
        IO.println(Wheel.getWheelCount());    // static: ask the CLASS; no object need
    }
}

class Wheel {
    double radius;
    static int wheelCount;

    Wheel(double radius) {
        this.radius = radius;
        wheelCount++;
    }

    double getCircumference() {          // instance method: has a "this"
        return 2 * Math.PI * this.radius;
    }

    static int getWheelCount() {         // static method: no "this", static data only
        return wheelCount;
    }
}
```

- **Instance method:** belongs to an object.
  - It has a `this` available, so it can read and write *that object's* fields.
  - `getCircumference()` uses `this.radius`.

- **Static method:** belongs to the class.
  - There is **no `this`**, so it can touch only static data and its own parameters.
  - No object is needed to call it.

This is also why the getter for a static field is itself static: you ask the class using `Wheel.getWheelCount()`, not any one wheel.

Pause and predict

```
class Wheel {
    double radius;

    static double area() {
        return Math.PI * radius * radius;
    }
}
```

What happens?

- **A.** It compiles. `area()` uses the radius of the most recently created Wheel
- **B.** Compile error: non-static variable `radius` cannot be referenced from a static context
- **C.** It compiles, and `radius` is `0.0` inside a static method
- **D.** It compiles, but crashes at run time with a `NullPointerException`

Build: a Die

Build: a Die

Let's build some programs for game night. First up: a die you can roll.

Design questions first for the Die class:

- What it knows (fields):
  - how many faces it has;
  - what it currently shows;
  - and where its random numbers come from.
- What it does (methods): roll.

`Random` does the rolling: `rng.nextInt(n)` gives `0` up to `n - 1`, so `1 + rng.nextInt(numFaces)` is a die.

```
import java.util.Random;

class Main {
    void main() {
        Die d6 = new Die(6);
        Die d20 = new Die(20);
        IO.println("d6:  " + d6.roll());
        IO.println("d20: " + d20.roll());
        IO.println("d6 still shows: " + d6.getState());
    }
}

class Die {
    private Random rng;      // a Die CONTAINS a Random: a field can hold an object
    private int numFaces;
    private int state;       // last roll; 0 = not yet rolled

    public Die(int numFaces) {
        this.rng = new Random();
        this.numFaces = numFaces;
        this.state = 0;
    }

    public int roll() {
        state = 1 + rng.nextInt(numFaces);
        return state;
    }

    public int getState() {
        return state;
    }
}
```

Notes:

- `roll()` is a method with real behavior: it changes the object's state *and* reports it.
- Each Die holds its own `state`: rolling `d6` doesn't touch `d20`.
- `rng` is a field that holds **an object**. A Die *contains* a Random. Classes are built using other classes.
- The file now starts with `import java.util.Random;`.
- Wouldn't it be nice if `new Die()` just meant a six-sided die?

Counting Objects

The constructor sees every beginning

How would you count how many `Tree` objects a program has created?

You *could* put a counter next to every `new Tree(...)` in the program.

We will surely miss one, somewhere, eventually.

Better: every new Tree runs the constructor, so count there, in a static field. One shared counter for the whole class:

```
class Main {
    void main() {
        Tree t1 = new Tree(31.4);
        Tree t2 = new Tree(62.8);
        Tree t3 = new Tree(15.7);
        IO.println("trees created: " + Tree.getNumberOfTrees());   // 3
    }
}

class Tree {
    private double circumference;
    private static int numberOfTrees = 0;

    public Tree(double circumference) {
        this.circumference = circumference;
        numberOfTrees++;              // the constructor sees every birth
    }

    public double getRadius() {
        return circumference / (2 * Math.PI);
    }

    public static int getNumberOfTrees() {
        return numberOfTrees;
    }
}
```

Two things to notice:

- The bookkeeping is **centralized**: no call site can forget, because construction is the count.
- The getter for a static field is itself static: you ask the class, `Tree.getNumberOfTrees()`, not any

one tree.

The Convention and the Contract

The convention, and the contract

The convention

You'll notice that in `Die` and `Tree` the fields are `private` and the methods are `public`.

That is the standard shape of a Java class:

- **fields `private`**: outsiders can't reach in
- **methods `public`**: outsiders use these as the "front door"

Why? We'll explain next time.

The contract

Autograded work, in labs, take-homes and exams, checks your class against its **specification**:

- exact class name,
- exact method names,
- exact parameter lists,
- and the return types.

Read the spec like a contract:

1. Write every field and every method header **first**, straight from the spec, before any bodies.
2. Compile. Fix names and types until it is exactly the spec.
3. *Then* fill in the bodies.

Take Home: MenuItem

We're doing some work for Boiler Burgers today. Let's write your first class from a spec.

A `MenuItem` is one item on the menu. It knows its name and its price, and the class as a whole keeps one shared count of how many items have ever been created.

Open `MenuItem.java` and complete it:

- **Fields.** A `String` name, a `double` price, and an `int` count. Two of them belong to each item. One belongs to the class. Make all three `private`.
- **`MenuItem(String name, double price)`**: stores both values and counts the new item. Remember `this.`.
- **`getName()`** and **`getPrice()`**: accessors.
- **`applyDiscount(double percent)`**: a mutator: `applyDiscount(10)` lowers the price by 10% of its current value, so 8.50 becomes 7.65.
- **`getItemCount()`**: a static method, because you ask the class, not any one item: `MenuItem.getItemCount()`.

This workspace has two files, the way real Java projects do:

- `MenuItem.java` is the class, and
- `Main.java` is the program that uses it. **Run** runs `Main`.

You only need to edit `MenuItem.java`.

The starter file already carries a Javadoc block on the class and on every method.

Press **Test** to run the tests.